

	Departamento: Ciencias de la Computación
	Carrera: Software Asignatura: Análisis y Diseño de Software NRC: 30723
	Apellidos y nombres del estudiante: Llumiyinga Moreno Jerson Montufar Alejandro Morillo Ariel

Sangolquí, a 26 de mayo de 2026

ANTECEDENTES

El presente proyecto surge de la necesidad de optimizar una aplicación de gestión de estudiantes (CRUD) desarrollada bajo el patrón arquitectónico MVC. En su concepción inicial, los sistemas de este tipo tienden a presentar un fuerte acoplamiento entre la lógica de negocio, los detalles de almacenamiento y la interfaz gráfica. Específicamente, el controlador del sistema acumulaba código repetitivo para las transacciones y validaciones, y dependía de sentencias condicionales tradicionales (como bloques if-else) para determinar el estado visual del formulario y las acciones permitidas por el usuario. Para solucionar estos problemas de escalabilidad y rigidez, se planteó una refactorización profunda basada en la separación de responsabilidades mediante la creación de capas de abstracción (Repository y Service). Como complemento indispensable para esta mejora arquitectónica, se propuso la integración de los patrones de diseño de comportamiento State y Template Method, buscando transformar un sistema rígido en una solución modular, limpia y altamente mantenible a largo plazo.

OBJETIVO

- Refactorizar la arquitectura de un sistema CRUD de estudiantes mediante la implementación de capas de abstracción y patrones de diseño de comportamiento, para mejorar la modularidad, reducir la duplicidad de código y optimizar el mantenimiento del software.
- Implementar el patrón Template Method y una capa de abstracción de datos: Definir un esqueleto estructurado para las operaciones generales del sistema utilizando una clase abstracta, delegando la persistencia y reglas operativas a las capas de Servicio y Repositorio para erradicar el código duplicado y aislar la interfaz de usuario.
- Aplicar el patrón State en el control de la interfaz gráfica: Extraer el comportamiento dinámico del formulario (estados de Creación y Edición) hacia clases separadas, con el fin de eliminar la dependencia de largas cadenas de condicionales y gestionar de forma segura qué acciones (Agregar, Actualizar o Eliminar) puede ejecutar el usuario en tiempo real.

DESARROLLO

1. Conceptos

1.1. Patron Template

Definición

Template Method es un patrón de diseño de comportamiento que define el esqueleto de un algoritmo en la superclase pero permite que las subclases sobrescriban pasos del algoritmo sin cambiar su estructura.

Características

- Se basa en el principio de "Inversión de Control" (Principio de Hollywood: "No nos llames, nosotros te llamaremos").
- Promueve la reutilización de código centralizando las partes invariables de un algoritmo.
- Utiliza clases abstractas donde el método "plantilla" suele ser final para evitar que las subclases alteren el orden de ejecución.

1.2. Patron State

Definición

State es un patrón de diseño de comportamiento que permite a un objeto alterar su comportamiento cuando su estado interno cambia. Parece como si el objeto cambiara su clase.

Características

- Elimina la necesidad de usar sentencias condicionales largas (if/else o switch/case) para controlar el comportamiento según el estado.
- Extrae el comportamiento asociado a un estado particular hacia clases separadas (cada estado es un objeto).
- Cumple con el principio de Responsabilidad Única (SRP) y Abierto/Cerrado (OCP) de SOLID.

2. Ejemplo implementado

En el diseño de este sistema CRUD, se aplicó una arquitectura en capas para garantizar una clara separación de responsabilidades. Específicamente, se logró aislar completamente la lógica de negocio y los detalles de almacenamiento de la interfaz de usuario. La capa Repository (a través de EstudianteRepositorioMemoria) encapsula el manejo de los datos, mientras que la capa Service (EstudianteServicioBasico) procesa las validaciones y reglas operativas. La entidad principal, Estudiante, transita entre estas capas sin acoplarse a la interfaz. La principal ventaja de esta abstracción es la alta flexibilidad y facilidad de mantenimiento. Al interactuar mediante interfaces (EstudianteRepositorio y EstudianteServicio), el sistema queda preparado para que, si en el futuro se requiere cambiar el almacenamiento en memoria por una base de datos real, la interfaz gráfica y el controlador no sufran ninguna modificación.

2.1 Diagrama PlantUML del patrón State Aplicado

```
@startuml
!theme plain
skinparam classAttributeIconSize 0

package controller.state {
    interface EstadoFormulario {
        + manejarAgregar(contexto: EstudianteController): void
        + manejarActualizar(contexto: EstudianteController): void
        + manejarEliminar(contexto: EstudianteController): void
        + configurarInterfaz(contexto: EstudianteController): void
    }

    class EstadoCreacion {
        + manejarAgregar(contexto: EstudianteController): void
        + manejarActualizar(contexto: EstudianteController): void
        + manejarEliminar(contexto: EstudianteController): void
        + configurarInterfaz(contexto: EstudianteController): void
    }

    class EstadoEdicion {
        + manejarAgregar(contexto: EstudianteController): void
    }
}
```

```

+ manejarActualizar(contexto: EstudianteController): void
+ manejarEliminar(contexto: EstudianteController): void
+ configurarInterfaz(contexto: EstudianteController): void
}
}

package controller {
  class EstudianteController {
    - vista: EstudianteView
    - servicio: EstudianteServicio
    - estadoActual: EstadoFormulario
    + EstudianteController(vista: EstudianteView, servicio: EstudianteServicio)
    + iniciar(): void
    + cambiarEstado(nuevoEstado: EstadoFormulario): void
    + getVista(): EstudianteView
    - inicializarEventos(): void
    - llenarCamposDesdeTabla(): void
    + actualizarTabla(): void
    + ejecutarAgregar(): void
    + ejecutarActualizar(): void
    + ejecutarEliminar(): void
  }
}

```

EstadoFormulario <|.. EstadoCreacion

EstadoFormulario <|.. EstadoEdicion

EstudianteController o--> "1" EstadoFormulario : estadoActual

note right of EstudianteController

El "Contexto" mantiene una referencia
al estado actual y delega los eventos
de la vista a este estado.

end note

@enduml

2.2 Diagrama PlantUML del patrón Template Method Aplicado

```
@startuml
!theme plain
skinparam classAttributeIconSize 0

package controller {
    abstract class AccionCRUDTemplate {
        # vista: EstudianteView
        + AccionCRUDTemplate(vista: EstudianteView)
        + <<final>> ejecutar(): void
        {abstract} # procesar(id: int): void
        {abstract} # getMensajeExito(): String
        {abstract} # actualizarInterfaz(): void
    }

    class "Clases Anónimas (Agregar, Actualizar, Eliminar)" as ClasesAnonimas {
        # procesar(id: int): void
        # getMensajeExito(): String
        # actualizarInterfaz(): void
    }

    class EstudianteController {
        + ejecutarAgregar(): void
        + ejecutarActualizar(): void
        + ejecutarEliminar(): void
    }
}

AccionCRUDTemplate <|-- ClasesAnonimas
EstudianteController ..> AccionCRUDTemplate : "instancia al ejecutar"

note left of AccionCRUDTemplate::ejecutar
    Método Plantilla:
    1. Captura ID
    2. procesar(id)
    3. actualizarInterfaz()
    4. Muestra mensajeExito()
    5. Captura Excepciones
end note
```

note right of ClasesAnonimas

Se implementan en tiempo
de ejecución dentro de
los métodos del controlador.

end note

@enduml

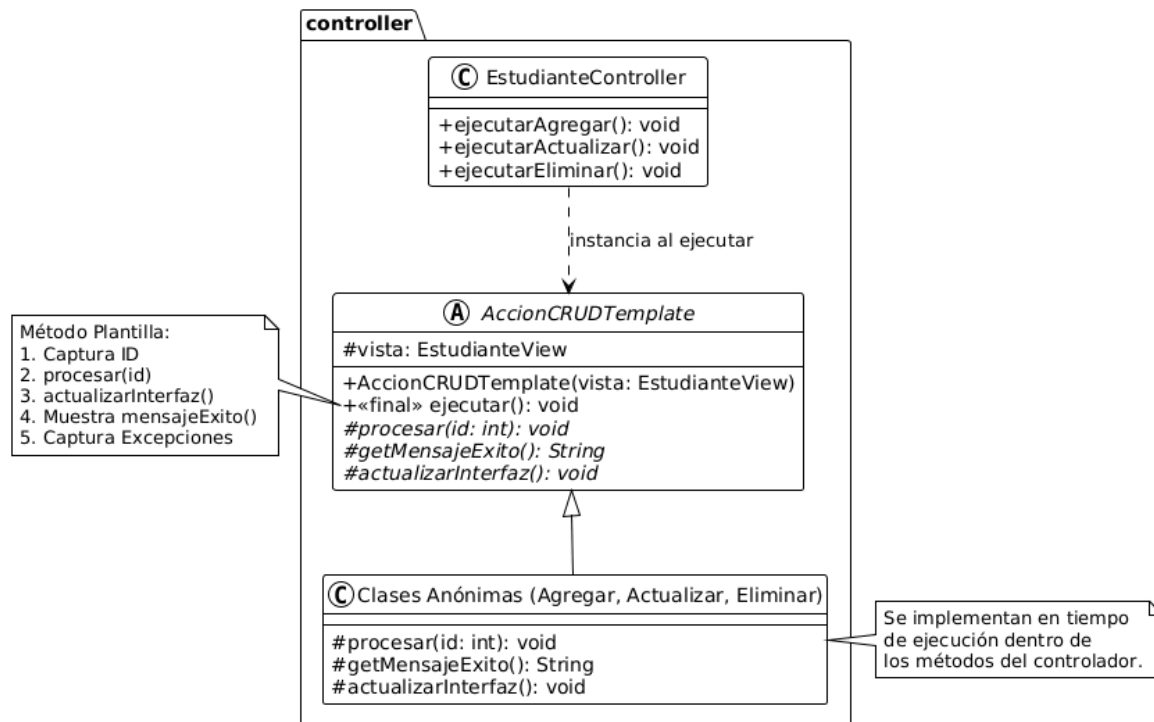


Ilustración 4. Modelo de Patrón Method Template

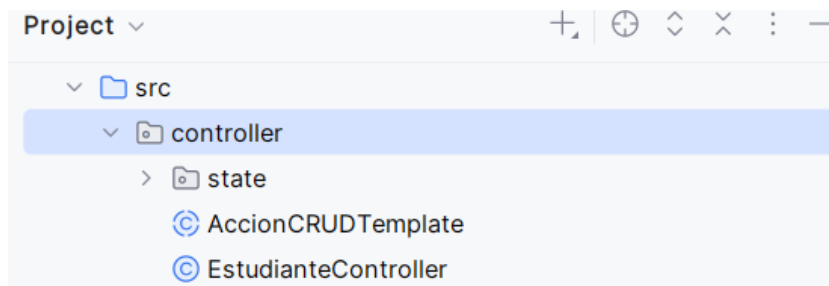
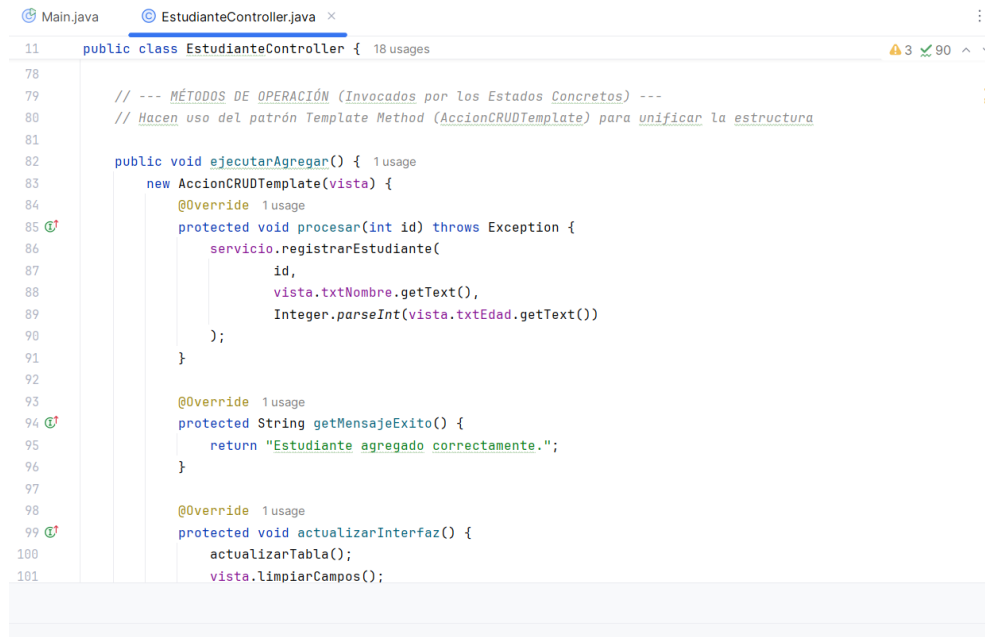


Ilustración 5. Implementación Patrón Template Method.



```
11 public class EstudianteController { 18 usages
78
79 // --- MÉTODOS DE OPERACIÓN (Invocados por los Estados Concretos) ---
80 // Hacen uso del patrón Template Method (AccionCRUDTemplate) para unificar la estructura
81
82 public void ejecutarAgregar() { 1 usage
83     new AccionCRUDTemplate(vista) {
84         @Override 1 usage
85         protected void procesar(int id) throws Exception {
86             servicio.registrarEstudiante(
87                 id,
88                 vista.txtNombre.getText(),
89                 Integer.parseInt(vista.txtEdad.getText())
90             );
91         }
92
93         @Override 1 usage
94         protected String getMensajeExito() {
95             return "Estudiante agregado correctamente.";
96         }
97
98         @Override 1 usage
99         protected void actualizarInterfaz() {
100             actualizarTabla();
101             vista.limpiarCampos();
102         }
103     }
104 }
```

Ilustración 6. Implementación Patrón Template Method.

3. Ventajas y desventajas

3.1. Patrón state

Ventajas

- Principio de responsabilidad única. Organiza el código relacionado con estados particulares en clases separadas.
- Principio de abierto/cerrado. Introduce nuevos estados sin cambiar clases de estado existentes o la clase contexto.
- Simplifica el código del contexto eliminando voluminosos condicionales de máquina de estados.

Desventajas

- Aplicar el patrón puede resultar excesivo si una máquina de estados sólo tiene unos pocos estados o raramente cambia.
- A veces los estados necesitan conocerse entre sí para hacer transiciones.

3.2. Patrón Template

Ventajas

- Puedes permitir a los clientes que sobrescriban tan solo ciertas partes de un algoritmo grande, para que les afecten menos los cambios que tienen lugar en otras partes del algoritmo.
- Puedes colocar el código duplicado dentro de una superclase.

Desventajas

- Algunos clientes pueden verse limitados por el esqueleto proporcionado de un algoritmo.
- Puede que violes el principio de sustitución de Liskov suprimiendo una implementación por defecto de un paso a través de una subclase.
- Los métodos plantilla tienden a ser más difíciles de mantener cuantos más pasos tengan.

4. ¿Por qué usar el patrón State?

El patrón State permite que un objeto altere su comportamiento cuando su estado interno cambia, pareciendo que el objeto cambia de clase. En este proyecto, se usa para controlar dinámicamente qué acciones puede realizar el usuario en la interfaz gráfica (UI) dependiendo de si está ingresando datos nuevos o modificando un registro existente, eliminando la necesidad de usar múltiples condicionales if-else.

Elemento del patrón State	Clase en el proyecto	Responsabilidad
Context	EstudianteController	Mantiene una referencia al estado actual de la vista y delega el manejo de los eventos de los botones a este objeto de estado.
State	EstadoFormulario	Define la interfaz común que encapsula el comportamiento asociado a los botones del formulario (Agregar, Actualizar, Eliminar).
Concrete State	EstadoCreacion	Implementa el comportamiento cuando el formulario está limpio. Habilita la creación y bloquea la edición/eliminación.
Concrete State	EstadoEdicion	Implementa el comportamiento cuando se selecciona un registro de la tabla. Habilita la edición/eliminación y bloquea la creación.

5. ¿Por qué usar el patrón Template Method?

El patrón Template Method define el esqueleto de un algoritmo en la superclase pero permite que las subclasses sobrescriban pasos específicos sin cambiar su estructura. En este proyecto, se utiliza para unificar el flujo de ejecución de las operaciones CRUD (validar, ejecutar, actualizar tabla, mostrar mensaje, atrapar errores), reduciendo drásticamente el código repetitivo.

Elemento del patrón Template Method	Clase en el proyecto	Responsabilidad
Abstract Class	AccionCRUDTemplate	Define el método plantilla ejecutar() que contiene el flujo estructurado y seguro, y declara los pasos abstractos.
Concrete Class	Clases Anónimas en EstudianteController	Implementan los detalles específicos (procesar, getMensajeExito, actualizarInterfaz) para las acciones de Agregar, Actualizar y Eliminar.

CONCLUSIONES

- En conclusión, la refactorización de este proyecto muestra que los patrones de diseño y la separación de capas no son solo conceptos teóricos, sino que brindan beneficios directos y tangibles durante la implementación.

- La separación de la abstracción, en particular, resultó muy beneficiosa. Al crear las capas de Servicio y Repositorio, se logró aislar completamente la lógica de negocio de los detalles de almacenamiento de datos. Esto facilitó el mantenimiento del sistema, ya que se volvió altamente modular. Por ejemplo, si se requiere cambiar el almacenamiento en memoria por una base de datos real en el futuro, ni la interfaz de usuario ni el controlador se verán afectados.
- Otro beneficio importante fue la reducción de código. La implementación del patrón Template Method en el controlador permitió definir un “esqueleto” para todas las operaciones del CRUD. Esto eliminó la necesidad de repetir la misma lógica en cada botón, lo que resultó en un código mucho más limpio y menos propenso a errores.
- La integración del patrón State también fue fundamental. Permitted eliminar las largas cadenas de condicionales que dictaban el comportamiento de la interfaz gráfica. Ahora, el formulario se autogestiona según su estado actual, lo que facilita agregar nuevas funcionalidades sin afectar el código existente.

En resumen, este ejercicio demuestra que la correcta separación de responsabilidades y el uso de patrones de comportamiento pueden transformar un sistema rígido en una solución flexible y mantenible a largo plazo.

RECOMENDACIONES

- Migración de Persistencia: Dado que la capa de abstracción de datos ya está correctamente implementada mediante interfaces, se recomienda desarrollar una nueva clase concreta (por ejemplo, `EstudianteRepositorioSQLServer`) para conectar el sistema a un motor de base de datos relacional.
- La arquitectura actual soportaría este cambio sin impactar las capas superiores. Escalabilidad del Patrón State: Se sugiere aprovechar la estructura del patrón State para incorporar nuevos comportamientos en futuras versiones. Por ejemplo, implementar un `EstadoSoloLectura` si el sistema evoluciona para manejar distintos roles de usuario con permisos limitados.
- Refinamiento del Template Method: Se recomienda ampliar los pasos del método plantilla en `AccionCRUDTemplate` para incluir un nivel más granular de manejo de excepciones, permitiendo registrar (loggear) los errores en un archivo de texto antes de mostrar la alerta en la interfaz.